Fiserv 2026

vSphere Supervisor Single-Host Bring-Up — Engineering Summary

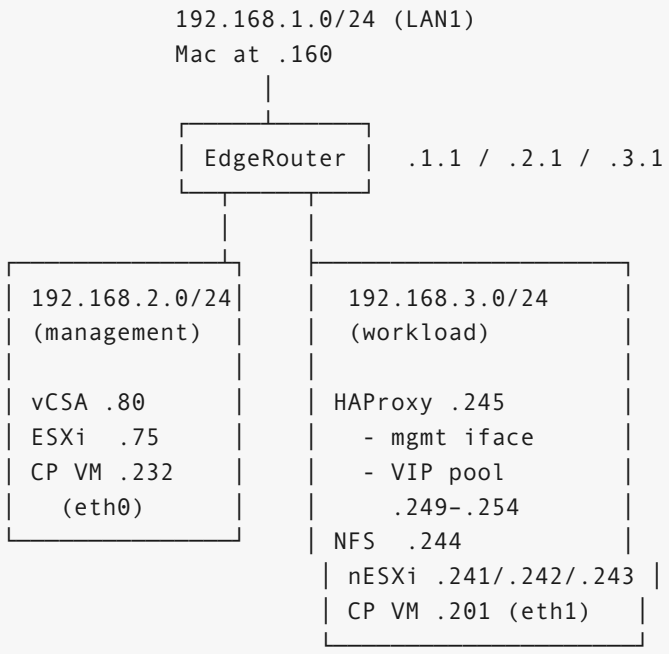
A condensed record of bringing up vSphere Supervisor (Kubernetes on vSphere) on a single physical ESXi host using nested ESXi. Captures the architecture, the eleven distinct root causes we hit, the commands that diagnosed and fixed each, and the helper scripts that make this reproducible.

> ***Scope:*** *this is the executive engineering summary. The full step-by-step runbook is in* `SUPERVISOR-INSTALL.md` *(Phases 1–12). This document assumes you've read that or are looking up a specific failure mode.*

Why this exists

vSphere Supervisor officially requires a cluster of at least 3 ESXi hosts. In our lab we have **one** physical host (`192.168.2.75`, 40 cores / 256 GB RAM). The workaround is to build a 3-host cluster of *nested* ESXi VMs on the one physical host and run Supervisor there. Unsupported by VMware; well-trodden in the community.

Final architecture



```
Physical host vSwitches/DVS:
  vSwitch0 → vmnic0 → 192.168.2.x (host mgmt)
  vSwitch1 → vmnic5 → 192.168.3.x (lab VM Network — promisc Accept)
  DSwitch  → vmnic4 → 192.168.2.x (bridge for nested-mgmt path)

Nested ESXi (.241/.242/.243) each have THREE vNICs:
  vmnic0 → outer VM Network (own vSwitch0 / vmk0)
  vmnic1 → outer VM Network (supervisor-dvs uplink1 → workload)
  vmnic2 → outer outer-mgmt-net (supervisor-dvs uplink2 → management)

supervisor-dvs (cluster-wide DVS across the 3 nested hosts):
  Port group "sup-workload"  active uplink = uplink1 → vmnic1 → 192.168.3.x
  Port group "sup-mgmt"      active uplink = uplink2 → vmnic2 → 192.168.2.x

CP VM has 2 vNICs:
  eth0 on sup-mgmt     → 192.168.2.232/24 (DNS, vCenter talk)
  eth1 on sup-workload → 192.168.3.201/24 (kubelet, pods, LB backend)
```

Glossary — outer, inner, pNIC, vNIC

These terms get used throughout. Worth pinning down up front.

Term	Meaning
pNIC	**Physical Network Interface Card.** The actual silicon on the motherboard or a PCIe card, with real RJ45 / SFP+ ports. ESXi names each one `vmnicN` regardless of who made it. From earlier inventory: this physical host has 8 pNICs (4 QLogic SFP+, 4 Broadcom RJ45), of which 3 have cables plugged in.
vNIC	**Virtual Network Interface Card.** A software NIC inside a VM, connected to a port group. Shows up as `ethN` inside the guest OS (or as `vmnicN` if the guest itself is ESXi).
outer	The *physical host's* layer — port groups, vSwitches, DVS, and pNICs that exist directly on the bare-metal ESXi at 192.168.2.75.
inner	The *nested ESXi cluster's* layer — port groups, DVS, and "vmnics" that exist inside the nested ESXi VMs. From the nested ESXi's perspective vmnicN looks physical, but it's really a vmxnet3 vNIC the outer host provided.

The two-layer structure looks like this:

```
═══════════════════════════════════════════════════════════════
  OUTER — physical host (192.168.2.75)
═══════════════════════════════════════════════════════════════

   Real pNICs:  vmnic0, vmnic1, ..., vmnic7   (real cables on real ports)

   Switches + port groups defined on this layer:
   - outer-mgmt-net      (carries 192.168.2.x — backed by pNIC vmnic4)
   - VM Network          (carries 192.168.3.x — backed by pNIC vmnic5)

   These are the "OUTER" port groups.
                            │
   VMs running on this host attach to these port groups via vNICs:
                            │
                            ▼
   ┌──────────────────────────────────────────────────────────────┐
   │  - nested-esxi-1  (a VM whose guest OS IS ESXi)              │
   │  - nested-esxi-2                                             │
   │  - nested-esxi-3                                             │
   │  - vCSA (vCenter Server Appliance)                           │
   │  - HAProxy VM, NFS VM                                        │
   └──────────────────────────────────────────────────────────────┘
                            │
        (each nested-esxi VM has 3 vNICs that attach to outer port
         groups; from the VM's guest OS those vNICs appear as
         vmnic0/vmnic1/vmnic2 "physical" interfaces)
                            │
                            ▼

═══════════════════════════════════════════════════════════════
  INNER — nested ESXi cluster (3 ESXi instances, each a VM)
═══════════════════════════════════════════════════════════════

   "pNICs" from the nested ESXi's view:  vmnic0, vmnic1, vmnic2
                                         (actually vNICs the outer
                                          host gave the VM)

   DVS + port groups defined on this layer:
   - supervisor-dvs                       (cluster-wide DVS)
     ├── sup-mgmt      → uplink2 (= inner vmnic2) → outer-mgmt-net
     └── sup-workload  → uplink1 (= inner vmnic1) → VM Network

   These are the "INNER" port groups.
                            │
                            ▼
   ┌──────────────────────────────────────────────────────────────┐
   │  VMs running on the nested cluster:                          │
   │    - SupervisorControlPlaneVM (CP VM, has 2 vNICs:           │
   │        eth0 → sup-mgmt, eth1 → sup-workload)                 │
   │    - Pod VMs                                                 │
```

```
    │   - Future TKG workload VMs                                          │
    └──────────────────────────────────────────────────────────────────────┘
```

Same network, two names. `outer-mgmt-net` and `sup-mgmt` are *both* on subnet `192.168.2.0/24` — same broadcast domain at the wire level — but each is configured separately because they live at different layers of the stack. Likewise `VM Network` and `sup-workload` share `192.168.3.0/24`.

Naming note. The outer management port group was originally created as `dswitch-vm` and renamed to `outer-mgmt-net` (May 2026); this document uses the current name throughout. In Terraform it is the `outer_dswitch_portgroup` variable (default `outer-mgmt-net`). The outer workload port group is the lab's pre-existing `VM Network` (`outer_vm_network_portgroup`); the name `outer-workload-net` is only used on fresh deploys where the `physical-network` module creates the port groups (`create_outer_networking = true`).

Why both layers need security-flag fixes. Phase 1 enabled the "Accept" security flags on the *outer* port groups. Phase 10 did the same on the outer `outer-mgmt-net` (which was missed initially). The *inner* port groups (`sup-mgmt`, `sup-workload`) have their own security flags that we also set to Accept. Skipping any layer drops nested-VM traffic silently.

What "bridge for nested management network" means

The phrase `outer-mgmt-net` "bridges to the management subnet" deserves unpacking, because it's the most subtle piece of the architecture.

"Bridge" here is the networking term — a connection that joins two L2 (Ethernet) segments so they behave as one broadcast domain. `outer-mgmt-net` is the bridge that gets traffic from *inside* a nested ESXi VM out to the physical `192.168.2.x` management network.

The CP VM doesn't run on the physical host — it runs *inside* a nested ESXi, which is itself a VM. So a packet from `CP VM eth0` to `192.168.2.1` has to traverse:

```
CP VM eth0 (.2.232)
  │
  ▼
supervisor-dvs port group "sup-mgmt"
  │  teaming says: egress only via uplink2
  ▼
uplink2 of each nested ESXi = its vmnic2
  │
  ▼
──────────── INSIDE → OUTSIDE the nested ESXi VM ────────────
  │
  ▼
Physical host's "outer-mgmt-net" port group
  │  (vmnic2 of nested-esxi-N is one of its ports)
  │  Security must Accept Forged Transmits, MAC Changes,
```

```
   │  Promiscuous — because the source MAC is the CP VM's,
   │  not vmnic2's own MAC.
   ▼
DSwitch (DVS on the physical host)
   │
   ▼
pNIC vmnic4 → physical Ethernet → EdgeRouter LAN2 port
   │
   ▼
192.168.2.1
```

From the **nested ESXi's** perspective, `vmnic2` is "a NIC plugged into the 192.168.2.x network." From the **physical host's** perspective, that same vmnic2 is "a port on outer-mgmt-net." The bridge is `outer-mgmt-net` stitching those two views together so they behave as one broadcast domain.

Why we needed it: without `vmnic2` / `outer-mgmt-net`, the only NICs on each nested ESXi were `vmnic0` and `vmnic1`, both attached to the outer `VM Network` port group on the 192.168.3.x lab subnet. There was no path from inside the nested cluster to the 192.168.2.x management network. Since the Supervisor wizard requires management and workload networks to be on *different* subnets, we extended each nested ESXi with a third vNIC bridged to `outer-mgmt-net` — giving the nested CP VM a way to put its management interface on 192.168.2.x.

Why "bridge" and not "route": routing means L3 forwarding between subnets; the two networks involved have different IP prefixes and a router decides which interface to send a packet out. A bridge is L2 — two segments merged into one broadcast domain with no L3 hop in between. Both `sup-mgmt` (inside the nested DVS) and `outer-mgmt-net` (on the physical host) are L2 segments on the *same* `192.168.2.0/24` network. The nested ESXi's vmnic2 just lets frames pass between them transparently.

HAProxy — what it is, why we need it, and how it must be set up

The Supervisor wizard requires a load balancer that fronts the K8s API server *and* programs frontends for every `Service{type: LoadBalancer}` users create. Without it, the wizard refuses to enable Supervisor. VMware supports two LB types: **NSX Advanced Load Balancer** (Avi) or **HAProxy**. We picked HAProxy because Avi requires a separate controller VM and licensing.

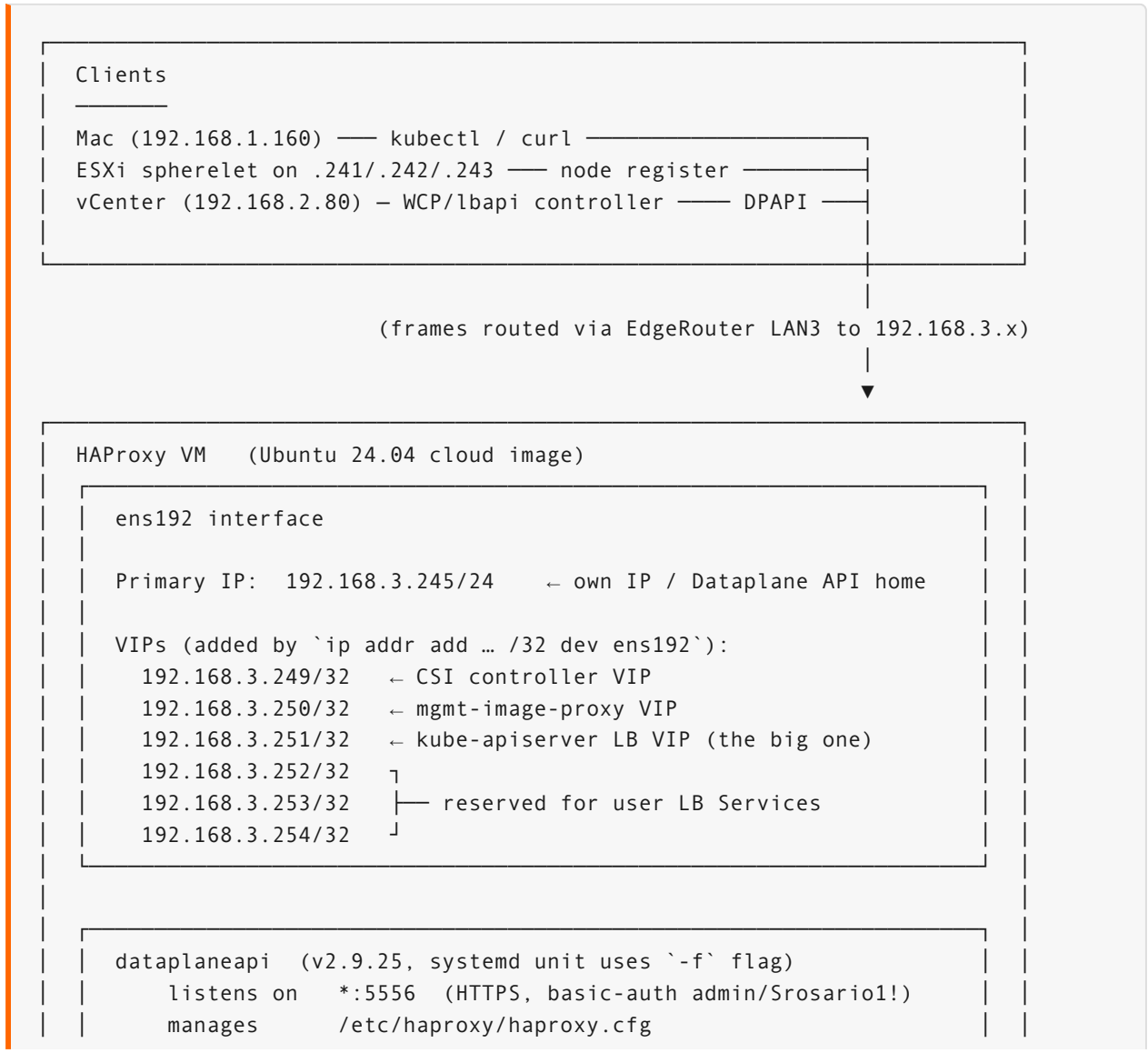
What HAProxy provides

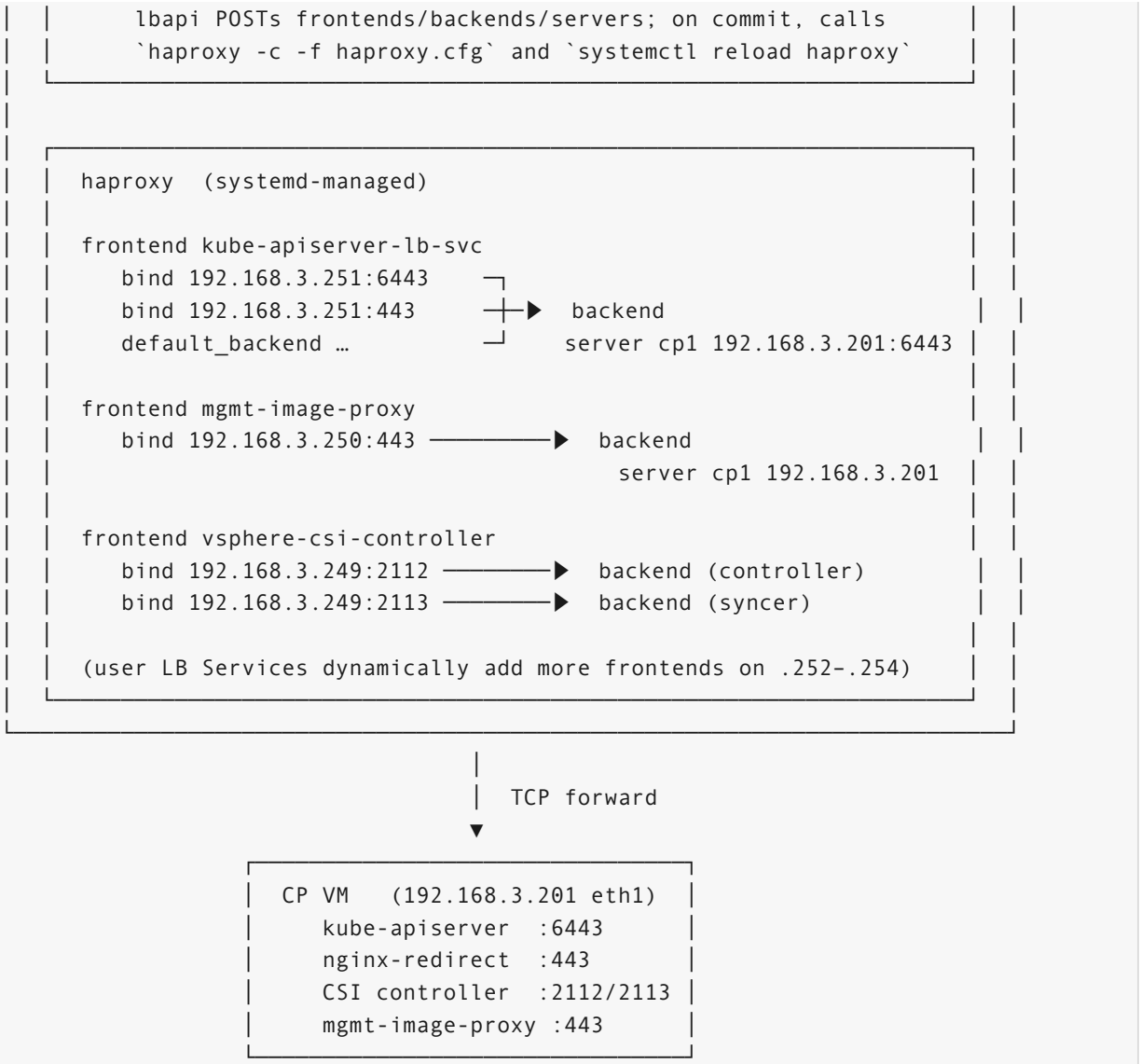
Function	Frontend	Backend
K8s API for kubectl clients	`192.168.3.251:6443`	CP VM workload IP `192.168.3.201:6443`

Function	Frontend	Backend
Plugin download / API HTTPS redirect	`192.168.3.251:443`	CP VM `192.168.3.201:443`
Supervisor mgmt-image-proxy	`192.168.3.250:443`	CP VM `192.168.3.201:443`
vSphere CSI controller	`192.168.3.249:2112` / `2113`	CP VM `192.168.3.201:2112` / `2113`
User LoadBalancer Services	from `.249-.254` pool	pod endpoint IPs

The frontends/backends/binds are programmed dynamically by the Supervisor's `vmware-system-lbapi` controller via the HAProxy Dataplane REST API — we don't hand-edit `haproxy.cfg`.

HAProxy VM — visual reference





IP-to-port reference for the HAProxy VM

IP	Port	Direction	Purpose
`192.168.3.245`	`5556`	inbound from vCenter	Dataplane API (HTTPS) — basic auth `admin:Srosario1!`
`192.168.3.245`	`22`	inbound from admin Mac	SSH to ubuntu user
`192.168.3.249`	`2112`	inbound	vSphere CSI controller LB → CP VM `.201:2112`
`192.168.3.249`	`2113`	inbound	

IP	Port	Direction	Purpose
			vSphere CSI syncer LB → CP VM `.201:2113`
`192.168.3.250`	`443`	inbound	Supervisor mgmt-image-proxy LB → CP VM `.201:443`
`192.168.3.251`	`6443`	inbound from kubectl	**K8s API LB** → CP VM `.201:6443`
`192.168.3.251`	`443`	inbound from browsers	nginx HTTPS redirect / plugin download → CP VM `.201:443`
`192.168.3.252-.254`	dynamic	inbound	reserved for user `Service{type:LoadBalancer}` VIPs

Requirements

1. **Standalone Ubuntu/Photon VM** dedicated to HAProxy (we used Ubuntu 24.04 cloud OVA).
2. **Two interfaces conceptually** — one for the Supervisor's *management* connection to the Dataplane API, one for the *workload* traffic that the VIPs serve. In our lab both happen to share the same `ens192` because everything is on `192.168.3.x` (workload subnet) and vCenter routes cross-subnet to reach the Dataplane API port.
3. **HAProxy + HAProxy Dataplane API** installed and listening on `:5556` over HTTPS with basic auth. The Supervisor wizard configures itself to talk to this endpoint.
4. **A TLS certificate** for the Dataplane API endpoint — self-signed is fine; the wizard pins it. Stored at `haproxy-dpapi.crt`.
5. **Outer port group must have all three security flags = Accept** (`VM Network` on vSwitch1 in our lab). HAProxy itself doesn't require this, but the nested ESXi → HAProxy traffic path does when the source MAC is a nested VM's.
6. **The VIPs must be claimed on an interface** — `net.ipv4.ip_nonlocal_bind=1` alone is **not enough** (see Root Cause #11). Each VIP from the configured pool needs `ip addr add <vip>/32 dev ens192`.
7. **The systemd unit must use the right flag** — `dataplaneapi -f /etc/haproxy/dataplaneapi.yaml`, NOT `--config-file=...` (see Root Cause #10).

Setup (minimum viable)

```
# 1. Build the VM
#    cloud-init: see haproxy-userdata.yaml (sets static IP .245,
#    creates ubuntu user, enables ip_nonlocal_bind)

#    Import via:
govc import.ova -options=<ova-spec.json> noble-server-cloudimg-amd64.ova

# 2. Run the installer (one-shot)
scp haproxy-setup.sh ubuntu@192.168.3.245:/tmp/
ssh ubuntu@192.168.3.245 'sudo bash /tmp/haproxy-setup.sh'

# The script:
#   - downloads HAProxy Dataplane API v2.9.25
#   - generates a self-signed TLS cert at /etc/haproxy/certs/dpapi.crt
#   - writes /etc/haproxy/haproxy.cfg with the minimal global+defaults
#   - writes /etc/haproxy/dataplaneapi.yaml with admin/Srosario1! basic auth
#   - installs the systemd unit using `-f` (the correct flag)
#   - starts haproxy and dataplaneapi

# 3. POST-INSTALL: claim VIPs on ens192
for ip in 192.168.3.249 192.168.3.250 192.168.3.251 \
          192.168.3.252 192.168.3.253 192.168.3.254; do
  sudo ip addr add $ip/32 dev ens192
done
# Persist via netplan so they survive reboot. The Terraform haproxy module
# bakes all VIPs into /etc/netplan/60-static.yaml via cloud-init; the manual
# bring-up used a separate 61-vips.yaml.

# 4. Verify
curl -sk -u admin:'Srosario1!' https://192.168.3.245:5556/v2/info
# Expect JSON with "version":"v2.9.25..."

# Manual transaction test (catches Root Cause #10 if it returns)
VER=$(curl -sk -u admin:'Srosario1!' \
  https://192.168.3.245:5556/v2/services/haproxy/configuration/version)
TX=$(curl -sk -u admin:'Srosario1!' -X POST \
  "https://192.168.3.245:5556/v2/services/haproxy/transactions?version=$VER")
TX_ID=$(echo "$TX" | python3 -c "import json,sys; print(json.load(sys.stdin)
        ['id'])")
curl -sk -u admin:'Srosario1!' -X POST -H 'Content-Type: application/json' \
  "https://192.168.3.245:5556/v2/services/haproxy/configuration/backends?
        transaction_id=$TX_ID" \
  -d '{"name":"test","mode":"tcp","balance":{"algorithm":"roundrobin"}}'
curl -sk -u admin:'Srosario1!' -X PUT \
  "https://192.168.3.245:5556/v2/services/haproxy/transactions/$TX_ID"
# Expect: {"status":"success",...}
```

Wizard inputs that target HAProxy

When you get to the wizard's "Load Balancer" page (Page 4), the values are:

Field	Value
Type	HAProxy
Name	`haproxy-lab`
Data Plane API Addresses	`192.168.3.245:5556`
User / Password	`admin` / `Srosario1!`
VIP Range	`192.168.3.249-192.168.3.254`
Server CA Certificate	paste contents of `haproxy-dpapi.crt`

Common HAProxy failure modes

Symptom	Cause	Fix
`failed to commit transaction: 400 Bad Request` in `lbapi` logs	systemd unit uses `--config-file=` (HAProxy config flag) instead of `-f` (dataplaneapi config flag)	Root Cause #10
`EXTERNAL-IP` stays `<pending>` for new LB Services	VIPs not claimed on `ens192`	Root Cause #11
`dataplaneapi.yaml` has lost indentation, all keys at top level	dataplaneapi v2.9.10 rewrote it as if it were `haproxy.cfg`	Root Cause #10 (also upgrade to ≥v2.9.25)
`curl https://.245:5556/v2/info` returns 404 or connection-refused	`dataplaneapi` service not running, or wrong listen port	`sudo systemctl status dataplaneapi`
`curl` to a VIP TCP-succeeds from same subnet but fails from outside	outer port group security policy blocks forged transmits	Root Cause #9

NFS storage — what it provides, why we need it, and how it was set up

Supervisor's HA placement and image cache need **shared storage** visible to every ESXi host in the cluster. We deployed a small dedicated Ubuntu VM on the *physical* cluster that exports a

single NFS share, and mounted it on each of the three nested ESXi hosts as the `nfs-shared` datastore.

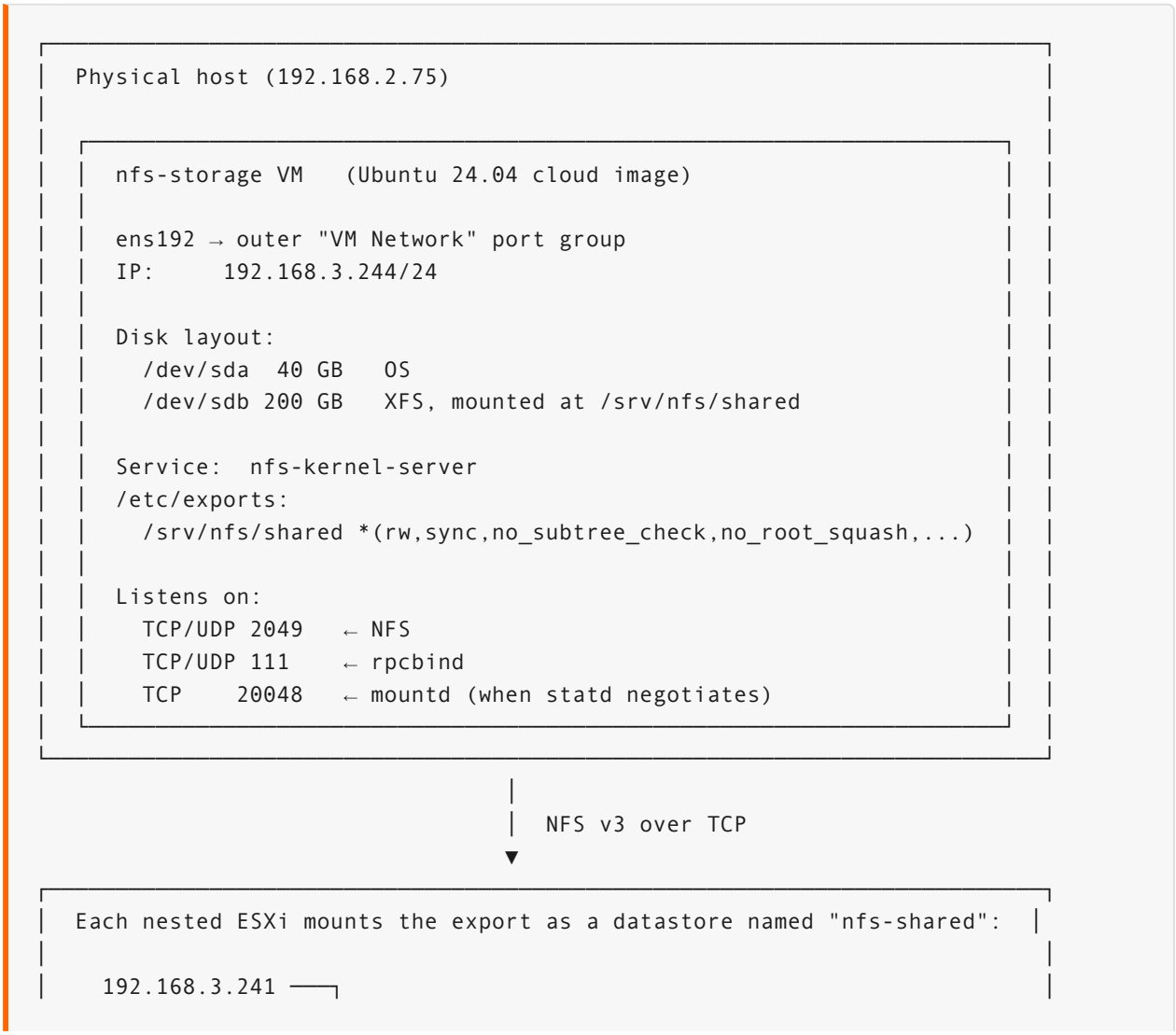
Why a separate VM (and not a nested-host local datastore)

Each nested ESXi VM has its own local datastore (the disk we attached in Phase 2). But local-only storage doesn't work for Supervisor because:

1. **Supervisor wants the same datastore on every host** so it can place control-plane VMs and Pod-VM images consistently. Local datastores are by-definition different per host.
2. **Putting the shared storage on one of the nested hosts** creates a dependency loop — if that host reboots, the storage goes away, which can make the host fail to come back up cleanly.

Solution: spin up an NFS server on the *outer physical* host, on its own datastore, separate from any nested ESXi. Every nested host mounts the export as a datastore named `nfs-shared`.

NFS architecture — visual reference



```
│    192.168.3.242 ──┬──→ nfs-shared (Remote NFS Volume)                  │
│    192.168.3.243 ──┘    Remote host:  192.168.3.244                    │
│                         Remote path:  /srv/nfs/shared                  │
│                         Access:       Read/Write                       │
│                                                                        │
│  Supervisor uses nfs-shared for:                                       │
│    - Control plane VM disk                                             │
│    - Pod VM ephemeral storage                                          │
│    - Image cache (kubeimage)                                           │
│    - PV provisioning when a PVC requests the supervisor-storage StorageClass │
└────────────────────────────────────────────────────────────────────────┘
```

Sizing and placement

Spec	Value	Why
OS	Ubuntu 24.04 cloud image	small, scripted via cloud-init
CPU	1–2 vCPU	NFS server is I/O-bound, not CPU
Memory	4 GB	comfortable for a 200 GB share with kernel NFS
OS disk	40 GB thin	fits Ubuntu + tools + room
Share disk	200 GB thin	the actual nfs-shared content
Placement	**physical** Cluster, NOT a nested host	avoids dependency loops
Network	`VM Network` on `vSwitch1` (outer)	same L2 the nested hosts use
IP	`192.168.3.244` (static, outside DHCP range)	predictable for mount cmds

Setup sequence (what Phase 6 of the runbook does)

```
# 1. Pull the Ubuntu cloud OVA
curl -fsSL -o /tmp/ubuntu-24.04-server-cloudimg-amd64.ova \
  https://cloud-images.ubuntu.com/releases/24.04/release/ubuntu-24.04-server-
        cloudimg-amd64.ova

# 2. Generate an import spec, set cloud-init guestinfo (static IP, user)
govc import.spec /tmp/ubuntu-24.04-server-cloudimg-amd64.ova > /tmp/nfs-spec.json
# (edit /tmp/nfs-spec.json: set guestinfo.userdata to base64-encoded cloud-init)

# 3. Import the OVA onto the physical host
```

```
govc import.ova -options=/tmp/nfs-spec.json \
  -dc=Datacenter -ds=datastore1 -name=nfs-storage \
  /tmp/ubuntu-24.04-server-cloudimg-amd64.ova

# 4. Power on; cloud-init runs and configures static IP + ubuntu user
govc vm.power -on=true /Datacenter/vm/nfs-storage

# 5. SSH in, grow the OS disk if needed, install NFS server
ssh ubuntu@192.168.3.244 'sudo bash -s' <<'OUTER'
# Grow the share disk to 200 GB if not already
parted -s /dev/sdb mklabel gpt mkpart primary xfs 0% 100% || true
mkfs.xfs -f /dev/sdb1
mkdir -p /srv/nfs/shared
echo '/dev/sdb1  /srv/nfs/shared  xfs  defaults  0  0' >> /etc/fstab
mount /srv/nfs/shared
chmod 0777 /srv/nfs/shared

apt-get install -y nfs-kernel-server
cat > /etc/exports <<EXPORTS
/srv/nfs/shared *(rw,sync,no_subtree_check,no_root_squash,insecure)
EXPORTS
systemctl enable --now nfs-kernel-server
exportfs -ra
OUTER

# 6. From vSphere Client (or govc), mount the export on each nested ESXi
#    UI: host → Datastores → New Datastore → NFS → 192.168.3.244:/srv/nfs/shared
#    Name on each host MUST be the same: "nfs-shared"
for h in 192.168.3.241 192.168.3.242 192.168.3.243; do
  govc datastore.create -type=nfs \
    -name=nfs-shared \
    -remote-host=192.168.3.244 \
    -remote-path=/srv/nfs/shared \
    "/Datacenter/host/Supervisor-Cluster/$h"
done

# 7. Verify each host sees the same datastore
for h in 192.168.3.241 192.168.3.242 192.168.3.243; do
  govc host.info -host "/Datacenter/host/Supervisor-Cluster/$h" \
    | grep -i nfs-shared || echo "$h: missing nfs-shared"
done
```

NFS export options explained

```
/srv/nfs/shared *(rw,sync,no_subtree_check,no_root_squash,insecure)
```

Option	Meaning
`*`	any client may mount (lab; tighten to subnet `192.168.3.0/24` for production)

Option	Meaning
`rw`	read-write
`sync`	writes are committed before the server acks (safer than `async` for VM datastores)
`no_subtree_check`	disables an expensive directory-membership check; recommended for modern NFS
`no_root_squash`	client's `root` keeps `root` privileges on the share — needed because ESXi mounts as `root`
`insecure`	allow client source ports ≥ 1024 (ESXi often uses high source ports)

Storage policy that ties Supervisor to this datastore

Supervisor wants storage referenced by a *vSphere Storage Policy*, not by datastore name. We create a tag-based policy that resolves to `nfs-shared` on any host that has it tagged:

```
# Create tag category + tag
govc tags.category.create -t Datastore supervisor
govc tags.create -c supervisor supervisor-storage

# Attach the tag to the nfs-shared datastore (on each host, but the
# tag is on the datastore object, so once is enough)
govc tags.attach supervisor-storage /Datacenter/datastore/nfs-shared

# Create the storage policy that selects datastores carrying this tag
govc storage.policy.create -e supervisor-storage \
  -t supervisor:supervisor-storage supervisor-storage
```

The wizard uses this policy name (`supervisor-storage` — the same tag, category, and policy names the Terraform `supervisor` module creates) for the Control Plane Storage Policy, Ephemeral Disks Storage Policy, and Image Cache Storage Policy on its Page 3.

NFS-specific failure modes

Symptom	Cause	Fix
`Permission denied` mounting from ESXi	`no_root_squash` missing in /etc/exports	add it, `exportfs -ra`

Symptom	Cause	Fix
`Mount point name already exists`	a stale `nfs-shared` mount with different remote path on some host	unmount the stale one first via `govc datastore.remove`
Different datastore on each host (one named `nfs-shared`, another `nfsshared`)	typo when adding via UI on one host	re-add with consistent name
Slow CP VM boot / pod-VM image pulls	`async` instead of `sync` and a flaky NIC	switch to `sync` (slower writes but predictable); check link state
`Stale file handle` after NFS server reboot	client cached handles invalidated	unmount + remount each host's datastore

Why we didn't use vSAN

vSAN would normally be the right answer for "shared storage across ESXi hosts," but it has a hard 3-host minimum and assumes each host contributes local storage. In our nested setup the nested hosts *could* run vSAN, but it would consume nested-host disk for vSAN cache+capacity tiers and add a layer of indirection vs the simple NFS-from-an-Ubuntu-VM approach. NFS gives us shared storage in ~10 lines of cloud-init and is sufficient for a lab.

Port reference (most-used)

From	To	Port	Purpose
Mac / admin	EdgeRouter	—	NAT / routing
Mac	vCSA `.2.80`	`443`	vSphere Client UI
Mac	vCSA `.2.80`	`22`	appliance shell
Mac	vCSA `.2.80`	`5480`	VAMI
WCP (in vCSA)	HAProxy `.3.245`	`5556`	Dataplane API
WCP (in vCSA)	CP VM `.2.232`	`6443`	kube-apiserver
`kubectl` clients	HAProxy VIP `.3.251`	`6443`	API via load balancer

From	To	Port	Purpose
`kubectl-vsphere`	HAProxy VIP `.3.251`	`443`	plugin download + login
ESXi spherelet → API	HAProxI VIP `.3.251`	`6443`	node registration
Anyone	NTP	UDP `123`	⚠️ **single most insidious dep**
CP VM → DNS	`.2.1` / `8.8.8.8`	UDP `53`	mgmt-net DNS validation

Port-group rationale (the L2 blueprint)

Port group	Lives on	Backing pNIC	Subnet	Security flags	Used by
`VM Network`	vSwitch1 (phys)	vmnic5	192.168.3.0/24	**all Accept**	nested ESXi vmnic0/ vmnic1, HAProxy, NFS
`outer-mgmt-net`	DSwitch (phys)	vmnic4	192.168.2.0/24	**all Accept**	nested ESXi vmnic2 (bridge to mgmt subnet)
`sup-workload`	supervisor-dvs	uplink1 (vmnic1)	192.168.3.0/24	active=uplink1	CP VM eth1, future workload VMs
`sup-mgmt`	supervisor-dvs	uplink2 (vmnic2)	192.168.2.0/24	active=uplink2	CP VM eth0

Why all three security flags must be `Accept` on the *outer* port groups (`VM Network`, `outer-mgmt-net`): nested ESXi forwards traffic whose source MAC is the inner VM's, not the outer vNIC's. With forged-transmits set to `Reject`, those frames are silently dropped — very hard to debug from inside.

Why the workload/management split: the Supervisor wizard requires management and workload networks on different subnets. Putting both on `192.168.3.x` (our original plan) gave the CP VM two routes to the same `/24` and the Linux kernel non-deterministically picked the wrong one for DNS queries — Phase 9 fix.

Eleven root causes hit, in order

1. vLCM depot version didn't match installed ESXi

Symptom: `eam.agent.install` failed with `Cannot download VIB spidev-esxio_0.1-1vmw.803.0.0.24022510.vib`.

Cause: the offline depot uploaded to vCenter Lifecycle Manager was 8.0U3 (build `24022510`) but the nested hosts were installed from the 9.0.2 ISO (build `25148076`). Supervisor enable puts the cluster into vLCM image-managed mode, which then tries to push the depot's VIBs to make hosts match — incompatible version.

Fix: uploaded the 9.0.2 offline depot via vSphere UI → Lifecycle Manager → Import → Bundle. After upload, the cluster's declared image still pointed at 8.0U3 (auto-generated when only that depot was available), so we also had to edit the image manually.

2. Cluster image declared the wrong base version

Symptom: depot was now correct, but compliance still showed "3 hosts incompatible".

Cause: the cluster's `autogen-software-spec` image was locked to `8.0 U3e - 24674464` from before the depot upload.

Fix (vSphere UI):

```
Cluster → Updates → Image → EDIT
  → ESXi Version dropdown → 9.0.2.0 - 25148076
  → VALIDATE → SAVE
```

Compliance flipped to "All hosts compliant" within seconds. Bonus: the stuck `disable` workflow that had been hanging at REMOVING for hours suddenly progressed once the image was satisfiable.

3. vCSA / physical host clock drift

The single most insidious failure. Symptom: every WCP request through the envoy proxy returned `Err <nil>` (empty error after a 2-minute timeout), and downstream operators reported missing CRDs/objects.

Cause: the physical host had **NTP disabled** by default, so its clock had drifted ~58 minutes behind real time. The vCSA inherited the drift via VMware Tools time sync. The nested ESXi VMs had NTP enabled and were on the correct time. CP VMs signed kube-apiserver certs at their (correct) current time; the vCSA saw `notBefore` as "in the future" and TLS handshakes silently failed.

Diagnose: from inside the vCSA shell —

```
echo | openssl s_client -connect 192.168.3.251:6443 -servername kubernetes 2>&1 \
  | grep -E 'notBefore|not yet valid'
# If you see "certificate is not yet valid" and a notBefore in the future,
# you have clock skew.
```

Fix:

One-shot helper (recommended — idempotent, does all 3 steps below):

```
./scripts/sv-fix-ntp                            # uses sv-env defaults
NTP_SERVER=162.159.200.1 ./scripts/sv-fix-ntp   # override NTP server
```

Or manually (govc + ssh):

```
HOST=/Datacenter/host/Cluster/192.168.2.75

# 1. Set NTP server (use IP, ESXi may have no DNS)
govc host.date.change -host "$HOST" -server 162.159.200.1

# 2. Enable + start ntpd  (host.service ignores -host without GOVC_HOST env)
GOVC_HOST=$HOST govc host.service enable ntpd
GOVC_HOST=$HOST govc host.service start  ntpd

# 3. Force vCSA system clock to catch up to its (now-correct) RTC
ssh root@192.168.2.80
> shell
hwclock --hctosys --utc
date -u   # should now match real time
```

WCP starts succeeding on its next retry cycle (~30s). All missing CRs (`VSphereDistributedNetwork`, `HAProxyLoadBalancerConfig`, `GatewayClass`) appeared within 2–3 minutes.

4. WCP's `service-control --restart` doesn't actually restart

Symptom: WCP wedged with no log activity; `service-control --restart wcp` reports success but PID doesn't change.

Cause: the soft restart path can silently no-op when wcpsvc has stuck file handles or is in a non-cancellable wait.

Fix: hard kill + start.

```
ssh root@192.168.2.80
> shell
service-control --stop wcp
sleep 3
pkill -9 -f wcpsvc 2>/dev/null
service-control --start wcp
ps -ef | grep wcpsvc | grep -v grep   # PID should now be new
```

The live WCP log is at `/storage/log/vmware/wcp/wcpsvc.log` (NOT `/var/log/vmware/wcp/wcpsvc.log` — that one stops being written after a restart).

5. vCSA Tiny size caused memory pressure

Symptom: WCP timeouts, slow API responses, swap-in-use 6+ GiB.

Cause: vCSA was deployed at Tiny (2 vCPU / 14 GiB RAM); WCP's state tracking pushed it past available headroom for the cluster reconcile cycles.

Fix (hot-add — no downtime):

```
. sv-env
VM='/Datacenter/vm/vCLS/vCenter-9-0'
govc vm.info -e=true "$VM" | grep -iE 'cpuHotAdd|memoryHotAdd'
# both should be true

govc vm.change -vm "$VM" -c=8 -m=32768   # 8 vCPU, 32 GiB

# Then on the vCSA itself, flush any pages still in swap:
ssh root@192.168.2.80
> shell
swapoff -a && swapon -a
```

6. CP VM had two interfaces on the same subnet

Symptom: `ManagementNetworkConfigured: FALSE`, "Unable to connect to the management DNS servers `192.168.2.1,8.8.8.8` from the control plane VM." DNS query times out at the local stub resolver.

Cause: wizard inputs put both management and workload on `192.168.3.0/24`. CP VM ended up with `eth0 = .3.232/24` and `eth1 = .3.201/24` — two equal-cost routes to the same `/24`. Linux kernel routed DNS replies out the wrong interface, breaking response path.

Fix: separate subnets. We hot-added a third vNIC to each nested ESXi connected to `outer-mgmt-net` (which bridges to LAN2), reconfigured `supervisor-dvs` to use that vmnic as uplink2, and pinned `sup-mgmt` port group to uplink2 only via teaming policy.

7. ESXi doesn't auto-detect hot-added PCI

Symptom: added the third vNIC via `govc vm.network.add` but `esxcli network nic list` still showed only `vmnic0/vmnic1` on each nested host.

Cause: ESXi's PCI subsystem doesn't fully scan on hot-add, even with vmxnet3. Soft "Reboot Guest" via VMware Tools wasn't enough either.

Fix: full power-cycle of each nested ESXi VM.

```
for vm in nested-esxi-1 nested-esxi-2 nested-esxi-3; do
  govc vm.power -off=true -force=true "/Datacenter/vm/$vm"
done
sleep 10
for vm in nested-esxi-1 nested-esxi-2 nested-esxi-3; do
  govc vm.power -on=true "/Datacenter/vm/$vm"
done
# Wait for connection state to flip connected and vmnic2 to appear in pnic list.
```

8. govc can't update existing DVS host members

Symptom: `govc dvs.add` returns "already a member of supervisor-dvs" for hosts that need an additional uplink.

Cause: govc's `dvs.add` is creation-only. There's no `dvs.host.change`.

Fix: call vSphere API directly via pyvmomi.

```
pip3 install pyvmomi --break-system-packages
python3 -c "
import ssl
from pyVim.connect import SmartConnect
from pyVmomi import vim
ctx = ssl.create_default_context(); ctx.check_hostname=False;
        ctx.verify_mode=ssl.CERT_NONE
si = SmartConnect(host='vcenter.skynetsystems.io',
                  user='administrator@vsphere.local', pwd='Srosario1!',
                  sslContext=ctx)
content = si.RetrieveContent()
dvs = next(n for dc in content.rootFolder.childEntity
           for n in dc.networkFolder.childEntity
           if isinstance(n, vim.DistributedVirtualSwitch) and n.name=='supervisor-
        dvs')
hosts = {}
for dc in content.rootFolder.childEntity:
    for cluster in dc.hostFolder.childEntity:
        if hasattr(cluster, 'host'):
            for h in cluster.host:
                if h.name in ('192.168.3.241','192.168.3.242','192.168.3.243'):
                    hosts[h.name] = h
import time
```

```
specs = [vim.dvs.HostMember.ConfigSpec(
    operation='edit', host=h,
    backing=vim.dvs.HostMember.PnicBacking(pnicSpec=[
        vim.dvs.HostMember.PnicSpec(pnicDevice='vmnic1'),
        vim.dvs.HostMember.PnicSpec(pnicDevice='vmnic2'),
    ])) for h in hosts.values()]
task = dvs.ReconfigureDvs_Task(spec=vim.DistributedVirtualSwitch.ConfigSpec(
    configVersion=dvs.config.configVersion, host=specs))
while task.info.state == vim.TaskInfo.State.running: time.sleep(2)
print(task.info.state)
"
```

Uplink port-group names are lowercase `uplink1`, `uplink2` (the labels "Uplink 1" with a space are *not* valid — the API enforces the underlying names).

9. `outer-mgmt-net` had default-reject security flags

Symptom: after splitting management onto `outer-mgmt-net`, CP VM still couldn't reach DNS on `192.168.2.1`. Ping from another VM in the same subnet *did* work, but ping from the Mac (via EdgeRouter) didn't.

Cause: `outer-mgmt-net`'s default security policy was Promiscuous=Reject, ForgedTransmits=Reject, MAC-Changes=Reject. Frames from a nested-CP-VM have a source MAC different from vmnic2's MAC — the outer DSwitch saw them as "forged transmits" and dropped them. Same fix we applied to VM Network back in Phase 1, just on a port group we'd missed.

Diagnose:

```
# Drop in /tmp/check.py
import ssl
from pyVim.connect import SmartConnect
from pyVmomi import vim
ctx = ssl.create_default_context(); ctx.check_hostname=False; ctx.verify_mode=ssl.
        CERT_NONE
si = SmartConnect(host='vcenter.skynetsystems.io',
                  user='administrator@vsphere.local', pwd='Srosario1!',
        sslContext=ctx)
for dc in si.RetrieveContent().rootFolder.childEntity:
    for n in dc.networkFolder.childEntity:
        if isinstance(n, vim.dvs.DistributedVirtualPortgroup) and n.name=='outer-
        mgmt-net':
            s = n.config.defaultPortConfig.securityPolicy
            print('Promiscuous:', s.allowPromiscuous.value)
            print('Forged TX:',   s.forgedTransmits.value)
            print('MAC Changes:', s.macChanges.value)
```

Fix: flip all three to True via pyvmomi (or in vSphere UI: `Networking → outer-mgmt-net → Edit → Security → all Accept → OK`).

10. Dataplane API systemd flag mis-set

Symptom: `lbapi` controller error `failed to commit transaction: 400 Bad Request`. LoadBalancer Services stuck at `EXTERNAL-IP: <pending>`.

Cause: our own `haproxy-setup.sh` used `--config-file=` on the dataplaneapi binary, pointing at `dataplaneapi.yaml`. But `--config-file=` is the **HAProxy** config file flag; the dataplaneapi's own config file flag is `-f`. So dataplaneapi treated its own YAML as the HAProxy config, added management headers, rewrote it without indentation (serializing into HAProxy's text format), and from then on every transaction snapshot was named `dataplaneapi.yaml.<txid>` and failed `haproxy -c` validation.

Fix: edit `/etc/systemd/system/dataplaneapi.service`:

```
[Service]
ExecStart=/usr/local/bin/dataplaneapi -f /etc/haproxy/dataplaneapi.yaml
```

Then restore both `dataplaneapi.yaml` (with proper indentation) and `haproxy.cfg` to clean defaults, clean stale state in `/tmp/haproxy/`, and `systemctl restart dataplaneapi`.

Verify a transaction commit works:

```
VER=$(curl -sk -u admin:'Srosario1!' https://192.168.3.245:5556/v2/services/
        haproxy/configuration/version)
TX=$(curl -sk -u admin:'Srosario1!' -X POST "https://192.168.3.245:5556/v2/
        services/haproxy/transactions?version=$VER")
TX_ID=$(echo "$TX" | python3 -c "import json,sys; print(json.load(sys.stdin)
        ['id'])")

curl -sk -u admin:'Srosario1!' -X POST -H 'Content-Type: application/json' \
  "https://192.168.3.245:5556/v2/services/haproxy/configuration/backends?
        transaction_id=$TX_ID" \
  -d '{"name":"test","mode":"tcp","balance":{"algorithm":"roundrobin"}}'

curl -sk -u admin:'Srosario1!' -X PUT "https://192.168.3.245:5556/v2/services/
        haproxy/transactions/$TX_ID"
# Expect: {"status":"success",...}
```

11. HAProxy bound to VIPs but kernel didn't claim them

Symptom: HAProxy frontends defined with VIPs `.249-.251`, `ss -ltn` showed sockets bound to those IPs, **but pinging the VIPs from anywhere failed silently.** Spherelet on each ESXi host couldn't reach the kube-apiserver LB.

Cause: `net.ipv4.ip_nonlocal_bind=1` (set in the haproxy cloud-init) lets a process *bind* to a non-local IP, but the kernel still won't respond to ARP for that IP unless it's actually configured on an interface. So no traffic ever reached HAProxy's listening socket.

The VMware HAProxy OVA handles this automatically; we missed it in our vanilla pivot.

Fix: assign each VIP as a `/32` secondary on HAProxy's `ens192`.

```
# Live (immediate):
for ip in 192.168.3.249 192.168.3.250 192.168.3.251 \
          192.168.3.252 192.168.3.253 192.168.3.254; do
  sudo ip addr add $ip/32 dev ens192
done

# Persistent across reboot — netplan:
sudo tee /etc/netplan/61-vips.yaml >/dev/null <<'NET'
network:
  version: 2
  ethernets:
    ens192:
      addresses:
        - 192.168.3.245/24
        - 192.168.3.249/32
        - 192.168.3.250/32
        - 192.168.3.251/32
        - 192.168.3.252/32
        - 192.168.3.253/32
        - 192.168.3.254/32
NET
sudo chmod 600 /etc/netplan/61-vips.yaml
```

Ping immediately returned. K8s API now reachable via the LB VIP.

> ***Terraform note:*** *the* `haproxy` *module bakes the VIPs directly into* `/etc/netplan/60-static.yaml` *via cloud-init, so on a Terraform-built HAProxy VM there is no separate* `61-vips.yaml`*.*

Why ARP makes `ip_nonlocal_bind` insufficient

`ip_nonlocal_bind=1` is a one-line kernel patch to `bind(2)`: it lets a process bind to an IP that isn't on any interface. That's literally all it does. It doesn't tell the kernel "I own this IP" — so when an ARP request arrives asking "who has `.249`?", the kernel's ARP responder walks its local IP table, sees `.249` isn't there, and **silently drops the request**.

Without an ARP reply, the requesting host (EdgeRouter in our case) never learns a MAC to address frames to. The IP packet sits queued for a few retries, then drops. HAProxy's listening socket never sees a packet because nothing ever made it to the host's NIC for that IP.

`ip addr add 192.168.3.249/32 dev ens192` is what actually claims the IP. It adds the address to the kernel's local IP table, after which:

1. The kernel's ARP responder replies to requests for `.249` with `ens192`'s MAC.
2. The kernel's IP layer delivers inbound packets to the socket bound there.
3. ICMP echo (ping) replies are sent automatically.

The `/32` mask matters: it claims the IP without also adding a duplicate route to `192.168.3.0/24` (which would create the same two-routes-same-subnet problem we hit in Phase 9).

For multi-node HAProxy in production, `keepalived` does this automatically across peers, plus it broadcasts a *gratuitous ARP* on takeover so neighbors update their caches faster. For a single lab HAProxy, the static `ip addr add` is sufficient.

Mechanism	What it does	Answers ARP?
`bind()` on a socket	accept connections on this IP/port	No
`ip_nonlocal_bind=1`	let `bind()` succeed on non-local IPs	No
`ip addr add X dev IFACE`	add X to kernel's local IP table	**Yes**
`keepalived` / VRRP	manage `ip addr add/del` + GARP across nodes	Yes

Recovery commands cheat-sheet

These are the commands you'll reach for most often. All assume `. sv-env` is sourced.

SSH to vCSA appliance shell

```
ssh root@192.168.2.80
# Password: Srosario1!
> shell                  # drop from "Command>" Appliance Shell into bash
```

The vCenter REST/UI public DNS hostname (`vcenter.skynetsystems.io`) **does not accept SSH** through the WAN firewall; you must use the internal LAN2 IP `192.168.2.80`.

Get the current CP VM root password

```
ssh root@192.168.2.80
> shell
/usr/lib/vmware-wcp/decryptK8Pwd.py
#   IP:  (blank when HA off)
#   PWD: <17-char password>
```

The password rotates whenever Supervisor is enable→disable cycled.

Tail live WCP logs

```
ssh root@192.168.2.80
> shell
tail -f /storage/log/vmware/wcp/wcpsvc.log
```

Restart WCP hard

```
ssh root@192.168.2.80
> shell
service-control --stop wcp
sleep 3
pkill -9 -f wcpsvc 2>/dev/null
service-control --start wcp
```

Hot-resize the vCSA

```
govc vm.change -vm '/Datacenter/vm/vCLS/vCenter-9-0' -c=8 -m=32768
```

Test the K8s LB endpoint from outside

```
curl -sk --max-time 3 https://192.168.3.251:6443/version
# {"kind":"Status","apiVersion":"v1",...} → API reachable
```

Drop a test workload to validate end-to-end

```
kubectl -n sandbox create deployment nginx --image=nginx
kubectl -n sandbox expose deployment nginx --port=80 --type=LoadBalancer
kubectl -n sandbox get svc                # wait for EXTERNAL-IP
curl http://<EXTERNAL-IP>/                # should return nginx welcome
```

Wizard Quick Reference — every value, every screen

Keep this open while clicking through **Workload Management** → **Get Started**. All values are post-fix (after every root cause we hit was resolved).

Pre-flight checklist (run BEFORE opening the wizard)

Check	Verify with	Expected
All clocks aligned	`sv-clocks`	Mac, ESXi host, vCSA within 1 sec
Physical host NTP enabled	`govc host.date.info -host /Datacenter/host/Cluster/192.168.2.75`	`NTP service status: Running`
Cluster image = 9.0.2.0.25148076	Cluster → Updates → Image	"All hosts compliant"
9.x depot uploaded	LCM → Imported Depots	`VMware-ESXi-9.0.2-25148076-depot.zip`
supervisor-dvs port groups exist	`govc find / -type n`	`sup-mgmt`, `sup-workload`
Each nested ESXi has vmnic0/1/2	per-host pnic list	all three vmnics present
sup-mgmt teaming = uplink2	pyvmomi check	active `['uplink2']`
sup-workload teaming = uplink1	pyvmomi check	active `['uplink1']`
`outer-mgmt-net` security all Accept	pyvmomi check	all three flags True
`VM Network` security all Accept	`govc host.portgroup.info ...`	all Yes
HAProxy data plane API responding	`curl -sk -u admin:'Srosario1!' https://192.168.3.245:5556/v2/info`	JSON with v2.9.25

Check	Verify with	Expected
HAProxy transaction commit works	manual transaction test	`{"status":"success"}`
HAProxy VIPs on `ens192`	`ip -br a show ens192`	`.245/24` + `.249-.254/32`
VIPs pingable from Mac	`ping .249..254`	all reply
NFS datastore mounted	`govc datastore.info` per host	`nfs-shared` accessible

IP plan (final)

Purpose	IP	Notes
Mac admin	`192.168.1.x` (LAN1)	gateway `.1.1`
vCSA	`192.168.2.80`	FQDN `vcenter.skynetsystems.io`
Physical ESXi	`192.168.2.75`	host vmk0
EdgeRouter LAN2 GW	`192.168.2.1`	+ DNS forwarder
CP VM mgmt	`192.168.2.231-235` (range)	wizard reserves; .232 used with HA off
EdgeRouter LAN3 GW	`192.168.3.1`	+ DNS forwarder
EdgeRouter DHCP	`.4-.200`	shrunk in Phase 7
nested ESXi	`.241/.242/.243`	host vmk0
nfs-storage	`.244`	NFS export
HAProxy	`.245`	+ VIPs `.249-.254/32`
VIP pool	`192.168.3.248/29` (`.249-.254`)	for K8s LB services
Workload range	`.201-.230`	wizard reserves; CP VM eth1 + workload pods

Page-by-page wizard values

Page 1 — vCenter & Network

Field	Value
vCenter	`vcenter.skynetsystems.io`
Network Stack	**vSphere Distributed Switch** (HAProxy mode)
Activation Mode	**Cluster Deployment**

Page 2 — Cluster

Field	Value
Compute Cluster	`Datacenter / Supervisor-Cluster`

Page 3 — Storage

Field	Value
Control Plane Storage Policy	`supervisor-storage` (tag-based policy → `nfs-shared`)
Ephemeral Disks Storage Policy	same
Image Cache Storage Policy	same

Page 4 — Load Balancer

Field	Value
Name	`haproxy-lab`
Type	**HAProxy**
Data Plane API	`192.168.3.245:5556`
User / Password	`admin` / `Srosario1!`
VIP Range	`192.168.3.249-192.168.3.254`
Server CA	paste contents of `haproxy-dpapi.crt`

Page 5 — Management Network

Field	Value
Mode	**Static**
Network	`sup-mgmt` (on supervisor-dvs)
Starting IP	`192.168.2.231`
Subnet Mask	`255.255.255.0`
Gateway	`192.168.2.1`
DNS	`192.168.2.1, 8.8.8.8`
NTP	`pool.ntp.org`

Page 6 — Workload Network

Field	Value
Services CIDR	`10.96.0.0/24` (default)
Pods CIDR	`10.244.0.0/20` (default)
Workload DNS	`192.168.3.1, 8.8.8.8`
Network Port Group	`sup-workload` (on supervisor-dvs)
Gateway	`192.168.3.1`
Subnet	`255.255.255.0`
IP Ranges	`192.168.3.201-192.168.3.230`

Page 7 — Control Plane Advanced

Field	Value
Size	**Tiny**
HA	**OFF** for lab (saves ~16 GiB RAM), **ON** for production
API Server DNS Names	blank

Page 8 — TKG / Content Library

Field	Value
Content Library	optional; attach later if needed

Page 9 — Review and Finish

Click **Finish**. Deploy takes ~15–30 min (HA off) or 30–45 min (HA on).

Wizard failure modes — quick triage

Wizard error	Root cause	Fix referenced
“Cannot download VIB …”	depot ≠ host version	#1
“3 hosts incompatible”	cluster image declares wrong base	#2
Deploy hangs at “Configured Mgmt Network”	clock skew	#3
`ManagementNetworkDNSServerConnectionFailed`	two NICs same subnet OR security policy reject	#6, #9
`Resource ... VSphereDistributedNetwork not found`	WCP didn’t push CRs (clock again)	#3
`failed to commit transaction: 400 Bad Request`	dataplaneapi flag wrong	#10
`EXTERNAL-IP <pending>` forever	VIPs not claimed on `ens192`	#11
Worker node “context deadline exceeded”	spherelet can’t reach API VIP — usually #11; sometimes long backoff (reboot nested ESXi)	
`Signature verification not found` for Velero/TKG	non-blocking; auto-retries	

Logging into the Supervisor

Standard path (kubectl-vsphere plugin)

```
# Download plugin (substitute darwin-amd64 / linux-amd64 / windows-amd64)
curl -kLo /tmp/plugin.zip https://192.168.3.251/wcp/plugin/darwin-amd64/vsphere-
        plugin.zip
unzip -d /tmp/plugin /tmp/plugin.zip
sudo install -m 0755 /tmp/plugin/bin/kubectl         /usr/local/bin/
sudo install -m 0755 /tmp/plugin/bin/kubectl-vsphere /usr/local/bin/

# Log in
kubectl vsphere login --server=192.168.3.251 \
  --insecure-skip-tls-verify \
  --vsphere-username=administrator@vsphere.local

# Use it
kubectl config use-context 192.168.3.251
kubectl get nodes
kubectl get pods -A
```

Breakglass admin (admin.conf from CP VM)

```
sv-cp-pwd
SSHPASS='<that password>' sshpass -e \
  scp root@192.168.2.232:/etc/kubernetes/admin.conf ~/sup-admin.conf
sed -i.bak 's|server: https://.*|server: https://192.168.3.251:6443|' ~/sup-
        admin.conf
export KUBECONFIG=~/sup-admin.conf
kubectl get nodes
```

Helper scripts (drop in `~/bin/`)

`sv-env`

```
#!/usr/bin/env bash
export GOVC_URL='vcenter.skynetsystems.io'
export GOVC_USERNAME='administrator@vsphere.local'
export GOVC_PASSWORD='Srosario1!'
export GOVC_INSECURE=true
```

`sv-state`

```
#!/usr/bin/env bash
. sv-env
echo "Supervisor:  $(govc namespace.cluster.ls -json)"
```

```
echo "CP VMs:      $(govc find /Datacenter/vm -type m -name 'SupervisorControlPlan
        eVM*' | wc -l)"
echo "HAProxy:     $(curl -sk -u admin:'Srosario1!' --max-time 4 \
  https://192.168.3.245:5556/v2/services/haproxy/configuration/backends \
  | python3 -c 'import json,sys;print(len(json.load(sys.stdin).get(\"data\",
        []))))') backends"
for h in 192.168.3.241 192.168.3.242 192.168.3.243; do
  cs=$(govc host.info -host "/Datacenter/host/Supervisor-Cluster/$h" -json \
    | python3 -c "import json,sys; print(json.load(sys.stdin)['hostSystems'][0]
        ['runtime']['connectionState'])")
  echo "Host $h:  $cs"
done
```

`sv-cp-pwd`

```
#!/usr/bin/env bash
expect <<'EOF'
set timeout 30
log_user 0
spawn ssh root@192.168.2.80
expect "assword:"; send "Srosario1!\r"
expect "Command>"; send "shell\r"; sleep 2
log_user 1
send "/usr/lib/vmware-wcp/decryptK8Pwd.py | grep -E '^(IP|PWD):'\r"
expect -re "PWD: .*"; sleep 1
send "exit\r"; expect "Command>"; send "exit\r"; expect eof
EOF
```

`sv-wcp-restart`

```
#!/usr/bin/env bash
expect <<'EOF'
set timeout 90
spawn ssh root@192.168.2.80
expect "assword:"; send "Srosario1!\r"
expect "Command>"; send "shell\r"; sleep 2
send "service-control --stop wcp; sleep 3; pkill -9 -f wcpsvc 2>/dev/null;
        service-control --start wcp\r"
expect "Successfully started"; sleep 2
send "exit\r"; expect "Command>"; send "exit\r"; expect eof
EOF
echo "wcp restarted."
```

`sv-clocks` — diagnostic (read-only)

Prints reference UTC, physical-host clock, and vCSA timedatectl side-by-side so you can spot drift quickly. Doesn't fix anything.

```
#!/usr/bin/env bash
. sv-env
```

```
echo "Reference (UTC):  $(date -u +%FT%TZ)"
echo "Physical ESXi:    $(govc host.date.info -host /Datacenter/host/Cluster/
        192.168.2.75 | awk '/Current date/{$1=""; $2=""; $3=""; $4=""; print $0}'
        | xargs)"
expect <<'EOF' 2>/dev/null | grep -E 'Local time|RTC'
spawn ssh root@192.168.2.80
expect "assword:"; send "Srosario1!\r"
expect "Command>"; send "shell\r"; sleep 1
send "timedatectl | head -3\r"; sleep 2
send "exit\r"; expect "Command>"; send "exit\r"; expect eof
EOF
```

`sv-fix-ntp` — the actual clock-skew fix (idempotent)

Wraps the three Root Cause #3 commands into one script:

1. `govc host.date.change … -server <ntp_ip>` on the physical host
2. `GOVC_HOST=… govc host.service enable / start ntpd`
3. `ssh root@<vcsa> ... hwclock --hctosys --utc` — pull the corrected RTC into the vCSA's running kernel

It reads the current state first and only changes what's not already correct. Re-running is safe. See `scripts/sv-fix-ntp` for the full source. Use:

```
./scripts/sv-fix-ntp                                # uses sv-env defaults
NTP_SERVER=162.159.200.1 ./scripts/sv-fix-ntp       # override
```

`sv-haproxy-config`

```
#!/usr/bin/env bash
H='https://192.168.3.245:5556'; U='admin:Srosario1!'
echo "=== backends ==="
curl -sk -u "$U" --max-time 4 "$H/v2/services/haproxy/configuration/backends" \
  | python3 -c "import json,sys;[print('  '+b['name']) for b in
        json.load(sys.stdin)['data']]"
echo "=== frontends ==="
curl -sk -u "$U" --max-time 4 "$H/v2/services/haproxy/configuration/frontends" \
  | python3 -c "import json,sys;[print('  '+b['name']) for b in
        json.load(sys.stdin)['data']]"
```

`sv-disable`

```
#!/usr/bin/env bash
. sv-env
govc namespace.cluster.disable -cluster Supervisor-Cluster
sv-wcp-restart      # often required to actually unstick the disable
echo "Disable submitted. Watch with sv-state."
```

Running workloads — two paths

There are two ways to run K8s workloads on this Supervisor. Pick based on what you're deploying.

Path A — Pods directly on the Supervisor (works today)

The Supervisor *is* a K8s cluster. Deploy pods/services straight into a **vSphere Namespace**. Each pod runs as a small CRX-based Pod VM on the nested ESXi hosts.

```
# Create a vSphere Namespace (UI: Workload Management → Namespaces → New,
# or via govc):
govc namespace.create -cluster=Supervisor-Cluster \
  -storage="<your-storage-policy>" sandbox

# Log in (kubectl-vsphere plugin from Phase 12):
kubectl vsphere login --server=192.168.3.251 --insecure-skip-tls-verify \
  --vsphere-username=administrator@vsphere.local
kubectl config use-context sandbox

# Deploy
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --port=80 --type=LoadBalancer
kubectl get svc nginx           # EXTERNAL-IP should be in 192.168.3.249-.254
curl http://<external-ip>/      # nginx welcome page
```

Validates: Supervisor → lbapi → HAProxy → VIP → pod end-to-end. We confirmed this in Phase 12.

Limitations of Path A: no traditional worker nodes (pods are CRX VMs scheduled directly on ESXi), no DaemonSets that need to run on every worker, many Helm charts that assume "regular" kubelet semantics don't work, some operators expect host-network or containerd internals that Pod VMs don't expose. Good for stateless web apps, demos, LoadBalancer testing. Not for "I want to run Cassandra with its custom CSI driver."

Path B — TKG / VKS Workload Cluster (NOT yet working in our lab)

The Supervisor's **vSphere Kubernetes Service** (VKS, formerly Tanzu Kubernetes Grid / TKG) provisions standalone kubeadm-based K8s clusters on demand. Each has its own CP + worker VMs.

> ***Status:*** *Phase 8 hit signature-verification errors for the* `tkg.vsphere.vmware.com` *and* `velero.vsphere.vmware.com` *Supervisor Services — they couldn't install because no TKG*

> *content library was attached. Path B is sketched out for future work; not yet validated end-to-end.*

Sketch of steps:

1. **Subscribe to a TKG content library:**

```
govc library.create -sub https://wp-content.vmware.com/v2/latest/lib.json
        \
  -sub-autosync=false -on-demand=true tkg-content
```

2. **Activate VKS** — Workload Management → Services → Add → vSphere Kubernetes Service → select that content library. Wait for status to flip to Active.

3. **Attach** the content library + VM classes to your vSphere Namespace.

4. **Apply** a `Cluster` (Cluster API) resource in that namespace:

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata: { name: my-cluster, namespace: sandbox }
spec:
  clusterNetwork:
    services: { cidrBlocks: [10.96.0.0/12] }
    pods:     { cidrBlocks: [192.168.0.0/16] }
    serviceDomain: cluster.local
  topology:
    class: tanzukubernetescluster
    version: v1.31.4---vmware.1-fips.1-tkg.1   # check VKS catalog
    controlPlane: { replicas: 1 }
    workers:
      machineDeployments:
        - class: node-pool
          name: workers
          replicas: 3
    variables:
      - { name: vmClass,      value: best-effort-small }
      - { name: storageClass, value: <your-storage-class> }
```

5. **Watch:**

```
kubectl -n sandbox get cluster,machine -w
# Provisioning takes ~10-20 minutes
```

Cluster YAML schema — required vs optional

Field	Required?	Notes
`metadata.name`	yes	DNS-1035 (lowercase, hyphen, no dots)
`metadata.namespace`	yes	must be an existing vSphere Namespace
`spec.clusterNetwork.services.cidrBlocks`	yes	one or more CIDRs inside the workload cluster
`spec.clusterNetwork.pods.cidrBlocks`	yes	one or more CIDRs
`spec.clusterNetwork.serviceDomain`	optional	defaults to `cluster.local`
`spec.topology.class`	yes	ClusterClass name (e.g. `tanzukubernetescluster`) — list with `kubectl get clusterclass -A`
`spec.topology.version`	yes	must be in the VKS catalog — list with `kubectl get tanzukubernetesreleases -A`
`spec.topology.controlPlane.replicas`	yes	1 for lab, 3 or 5 for HA
`spec.topology.workers.machineDeployments[]`	yes (≥1)	each pool needs `class`, `name`, `replicas`
`spec.topology.variables[]`	depends on ClusterClass	usually at least `vmClass`, `storageClass`

What the YAML "is" — the ClusterClass model

The `topology.class` field references a **pre-installed ClusterClass** that VMware ships with the VKS service. The ClusterClass contains templates for every underlying Cluster API object — the `KubeadmControlPlane`, `VSphereCluster`, `VSphereMachineTemplate`, `KubeadmConfigTemplate`, addon `ClusterResourceSet`s, etc. The declarative `Cluster` you submit is *just* the high-level shape; the ClusterClass controller expands it into ~15 underlying CAPI objects that actually provision VMs and join them via kubeadm.

You typically only edit the `Cluster` object. Scaling workers? Change `replicas`. Upgrading K8s? Change `version`. The controllers reconcile everything underneath. Inspect what's actually running with:

```
kubectl -n sandbox get cluster,machine,vspheremachine,virtualmachine
```

Common ClusterClass variables (`spec.topology.variables[]`)

For VMware's `tanzukubernetescluster` ClusterClass:

Variable	Purpose
`vmClass`	VM shape (CP + workers). `best-effort-small`, `best-effort-medium`, `guaranteed-small`, etc. — list with `kubectl get virtualmachineclass -n <ns>`
`storageClass`	StorageClass for in-cluster PVs
`defaultStorageClass`	Mark a class as the default for the workload cluster
`nodePoolVolumes`	Extra disks attached to worker pools
`ntp`	NTP servers for nodes (defaults work)
`proxy`	http_proxy/https_proxy for air-gapped labs
`trust`	Extra CA bundles to inject into nodes

Use `kubectl describe clusterclass tanzukubernetescluster -n vmware-system-tkg` to see the full variable schema your VKS install exposes.

Minimum viable Cluster

```
apiVersion: cluster.x-k8s.io/v1beta1
kind: Cluster
metadata: { name: tiny, namespace: sandbox }
spec:
  clusterNetwork:
    services: { cidrBlocks: [10.96.0.0/12] }
    pods:     { cidrBlocks: [192.168.0.0/16] }
  topology:
    class: tanzukubernetescluster
    version: v1.31.4---vmware.1-fips.1-tkg.1
    controlPlane: { replicas: 1 }
    workers:
```

```
      machineDeployments:
        - { class: node-pool, name: w, replicas: 1 }
    variables:
      - { name: vmClass,      value: best-effort-small }
      - { name: storageClass, value: <your-storage-class> }
```

A 2-VM cluster (1 CP + 1 worker). Useful for testing the path end-to-end before committing larger resources.

Day-2 operations

```
# Scale workers
kubectl -n sandbox patch cluster my-cluster --type merge -p '
spec:
  topology:
    workers:
      machineDeployments:
        - { class: node-pool, name: workers, replicas: 5 }'

# Upgrade K8s
kubectl -n sandbox patch cluster my-cluster --type merge -p '
spec: { topology: { version: v1.32.1---vmware.1-fips.1-tkg.1 } }'

# Delete the cluster (cleans up all underlying VMs)
kubectl -n sandbox delete cluster my-cluster
```

1. **Log in** to the workload cluster:

   ```
   kubectl vsphere login \
     --server=192.168.3.251 --insecure-skip-tls-verify \
     --tanzu-kubernetes-cluster-namespace=sandbox \
     --tanzu-kubernetes-cluster-name=my-cluster \
     --vsphere-username=administrator@vsphere.local
   kubectl config use-context my-cluster
   kubectl get nodes
   ```

When Path B is attempted, validate in order:

1. CP VM can reach `wp-content.vmware.com:443` over HTTPS (workload net needs WAN egress via EdgeRouter — likely the *first* thing to break in an air-gapped lab)
2. Content library finished syncing (no “Syncing” state in UI)
3. VKS service shows **Active**, not signature-verification errors
4. `kubectl get tanzukubernetesreleases` on Supervisor lists K8s versions
5. `kubectl get clusterclass -A` shows `tanzukubernetescluster` class
6. Cluster resource transitions `Provisioning → Provisioned`
7. Machines reach `Running` state

The most likely failure points (from what we've seen): WAN egress from the workload network, and signature verification — both we've already hit once.

Lessons learned

1. **Clock first.** Before any TLS/cert/handshake debugging, verify every host in the path has the same view of "now" to the second. Clock skew in the path of a TLS handshake fails *silently*.
2. **Two `/24`s for two networks.** Even though "they're both `.3.x`, what's the harm" — Linux can't disambiguate two interfaces on the same subnet without source-routing rules. The wizard *requires* different subnets for a reason.
3. **All-Accept on every outer port group that backs nested traffic.** Forged transmits in particular: any time a nested VM sends frames out via a parent vNIC, the source MAC is the nested VM's, not the parent's. Default Reject = silent drop.
4. `service-control --restart` **is sometimes a no-op.** Capture PID before/after; if unchanged, `pkill -9` + `--start`.
5. **The live log isn't always where you think.** WCP writes `/storage/log/vmware/wcp/wcpsvc.log` after restart, not `/var/log/...` The latter goes stale.
6. **Read the CLI flag descriptions, not the names.** `--config-file=` on dataplaneapi means *HAProxy* config, not the daemon's own. The name is misleading. Always run `--help`.
7. `net.ipv4.ip_nonlocal_bind=1` **≠ "the VIP works".** It only enables the bind syscall. The kernel still won't answer ARP for the VIP unless the IP is configured on an interface (`ip addr add`).
8. **govc has gaps; pyvmomi fills them.** DVS host member updates, port-group security flags, and detailed object collection are easier from `pyvmomi`'s direct `Reconfigure*_Task` calls than from govc shell-outs.
9. **Document as you go.** This document and `SUPERVISOR-INSTALL.md` were maintained throughout. Each new finding became a new numbered phase. By the time we hit the eleventh distinct root cause, we could cross-reference earlier diagnoses (e.g., "Phase 1's security-flag fix on VM Network, but on outer-mgmt-net this time").

Automating future deploys — Terraform module

A working Terraform module that reproduces this entire bring-up lives at the **repo root** (this repo *is* the module). It encodes every Phase 1–11 workaround declaratively (NTP, port-group security

flags, supervisor-dvs with two uplinks, teaming policy, HAProxy with the right systemd flag and VIPs claimed). Layout:

```
.
├── README.md
├── versions.tf, variables.tf, main.tf, outputs.tf
├── modules/
│   ├── physical-network/ ← optional: outer vSwitches + port groups
(create_outer_networking=true)
│   ├── host-config/      ← NTP on physical host, outer port-group security flags
│   ├── network/          ← supervisor-dvs, sup-mgmt, sup-workload, uplinks,
teaming
│   ├── nested-esxi/      ← optional: build the nested ESXi VMs from the installer
ISO
│   ├── haproxy/          ← HAProxy VM with cloud-init + Phase 10/11 fixes baked
in
│   ├── nfs/              ← NFS storage VM (Ubuntu cloud image) + nfs-shared
datastore mounts
│   ├── content-library/  ← optional: subscribed TKG content library (tkg-content)
│   └── supervisor/       ← storage policy (supervisor-storage) + Supervisor
enable
└── examples/lab/         ← consumable example with the lab's IP plan
```

Usage:

```
cd examples/lab
cat > secrets.auto.tfvars <<EOF
vcenter_password = "<sso admin password>"
haproxy_password = "<dataplane api basic-auth password>"
EOF
chmod 600 secrets.auto.tfvars

terraform init
terraform plan
terraform apply   # ~25 min unattended, including supervisor enable
```

What it does *not* cover: - vCenter bootstrap (assumes existing) - vSphere SSO RBAC for namespace permissions

(Nested ESXi VM creation and the TKG content library subscription — formerly manual — are now covered by the optional `nested-esxi` and `content-library` modules.)

Trust-but-verify: this module reproduces the manual steps we documented, but hasn't been run end-to-end against a clean lab — the HAProxy and supervisor modules in particular have small drift points (govc CLI quirks, vSphere enable spec schema between versions). If something rejects on first apply, the runbook phase numbers are cross-referenced inline in each module's comments so you can dive in.

File index

File	Contents
`SUPERVISOR-INSTALL.md`	Full step-by-step runbook (Phases 1–12, ~3000 lines)
`SUPERVISOR-SUMMARY.md`	This executive summary
`haproxy-setup.sh`	Bootstrap script for the HAProxy VM (Dataplane API + TLS)
`haproxy-userdata.yaml`	cloud-init for the HAProxy Ubuntu base image
`haproxy-dpapi.crt`	Dataplane API TLS cert (pasted into Supervisor wizard)
`INSTALL.md` / `INSTALL.pdf`	Original Graylog install runbook (pre-Supervisor work)
`VCENTER-SESSION.md` / `.pdf`	Networking and vCenter setup play-by-play
`main.tf` / `variables.tf` / `modules/` / `examples/lab/`	Terraform module (at the repo root) that reproduces this bring-up declaratively
`build-pdf.sh`	Re-render this summary as Fiserv-styled PDF
`scripts/sv-*`	Operational helper scripts (sv-state, sv-clocks, sv-wcp-restart, …)